TPO POO – Etapa 1: Análisis y Diseño · Agencia de Turismo «Rumbo Sur»

Oct 29, 2026 · @Lucas Iacobellis

1. Propuesta seleccionada e integrantes

El grupo eligió la **Propuesta B: Agencia de Turismo «Rumbo Sur»**: un sistema que ordena la venta de tours, la operación de cada salida y la liquidación a guías, hoy manejadas en planillas.

Integrante	Legajo	Rol principal
Lucas Iacobellis	1217175	Modelado de casos de uso y secuencia
Facundo Fulop	1216428	Diagrama de clases y patrones

Materia: Programación Orientada a Objetos – UADE · **Repositorio:** https://github.com/LucasGI/TPO.POO- · **Entrega Etapa 1:** 25/10/2026 (tag `etapa-1`).

Herramientas: PlantUML (fuentes en `/docs/uml`), Java 17+, Git y GitHub.

2. Requisitos y supuestos

El sistema cubre siete áreas funcionales con seis actores; las reglas de negocio del enunciado se respetan todas y los valores ajustados quedan como supuestos.

2.1 Actores

Actor	Descripción	Interacción principal
Vendedor	Empleado con comisión sobre ventas	Registra clientes, crea, modifica y cancela reservas, registra pagos
Coordinador de operaciones	Empleado que opera las salidas	Programa salidas, asigna guía y vehículo, emite lista de pasajeros, inicia y cierra salidas
Administrador	Responsable del negocio	Administra catálogo, tarifas, personal y proveedores; liquida honorarios; emite reportes

Actor	Descripción	Interacción principal
Guía	Empleado que conduce las salidas	Consulta la lista de pasajeros y restricciones (actor secundario)
Cliente	Titular de la reserva	No opera el sistema: es atendido por el vendedor (se modela como actor secundario)
Reloj del sistema	Proceso temporal	Dispara la cancelación por cupo mínimo y la anulación por saldo impago

2.2 Requerimientos funcionales

Código	Requerimiento
RF01	Alta, baja y modificación de tours con categoría, itinerario de actividades ordenadas por día, cupo mínimo y máximo.
RF02	Registrar tarifas por temporada (baja, media, alta) con fecha de vigencia y precio base por pasajero.
RF03	Programar salidas de un tour con fecha de inicio, fecha de fin y cupo máximo.
RF04	Registrar clientes (nombre, apellido, documento, teléfono, correo, país).
RF05	Crear reservas para una salida con uno o más pasajeros, asociadas a un cliente titular y a un vendedor.
RF06	Calcular el precio de la reserva según tipo de pasajero, temporada, promociones y servicios adicionales.
RF07	Registrar la seña y los pagos del saldo, en cuotas y con distintos medios de pago.
RF08	Contratar servicios adicionales por pasajero (excursiones, traslados, seguro de viaje).
RF09	Cancelar reservas aplicando la política de cancelación y registrar la devolución.

Código	Requerimiento
RF10	Modificar reservas: agregar o reemplazar pasajeros.
RF11	Asignar guía y vehículo a cada salida, validando disponibilidad, habilitación y capacidad.
RF12	Emitir la lista de pasajeros con documento, contacto de emergencia y restricciones.
RF13	Iniciar, finalizar y cerrar salidas; liquidar los honorarios del guía.
RF14	Emitir reportes: ocupación por salida, ventas por vendedor, ingresos por tour y cancelaciones.
RF15	Persistir los datos en archivos entre ejecuciones.

2.3 Reglas de negocio

Código	Regla
RN01	Categorías de tour: aventura, cultural y naturaleza. Aventura exige edad mínima y nivel de dificultad.
RN02	Estados de salida: programada → confirmada → completa → en curso → finalizada; cancelada desde programada, confirmada o completa.
RN03	Una salida se confirma al alcanzar el cupo mínimo; si 7 días antes no lo alcanza, se cancela con devolución del 100 %.
RN04	Al llegar al cupo máximo la salida pasa a «completa» y no admite reservas.
RN05	Un guía no puede tener salidas superpuestas y solo guía categorías habilitadas. La capacidad del vehículo debe ser ⩾ cupo de la salida.
RN06	Una reserva ocupa tantos lugares como pasajeros tenga.
RN07	Tarifa por pasajero: adulto 100 %, menor de 12 años 60 %, menor de 3 años 0 % (se registra igual), jubilado 10 % de descuento.

Código	Regla
RN08	Promociones no acumulables: reserva anticipada (> 60 días) 15 %, grupo (⩾ 6 pasajeros) 10 %; se aplica la más favorable.
RN09	Los adicionales se suman por pasajero. El seguro de viaje es obligatorio en tours de aventura.
RN10	La reserva se confirma con una seña del 30 %. El saldo vence 10 días antes de la salida; si no se paga, la reserva se anula y libera cupo.
RN11	Pago con tarjeta en cuotas: recargo del 7 %.
RN12	Devolución por cancelación del cliente: ⩾ 30 días 90 %; 29–15 días 50 %; 14–7 días 20 %; < 7 días 0 %. Si cancela la agencia: 100 %.
RN13	Un pasajero puede reemplazarse sin cargo hasta 3 días antes, si cumple las condiciones del tour.
RN14	No se inicia una salida sin lista de pasajeros emitida ni guía asignado.
RN15	Honorario del guía por salida finalizada: monto fijo por día + 5 % de las ventas de adicionales de esa salida.
RN16	Una reserva pagada no se elimina: las correcciones se registran como notas de ajuste.

2.4 Supuestos adoptados

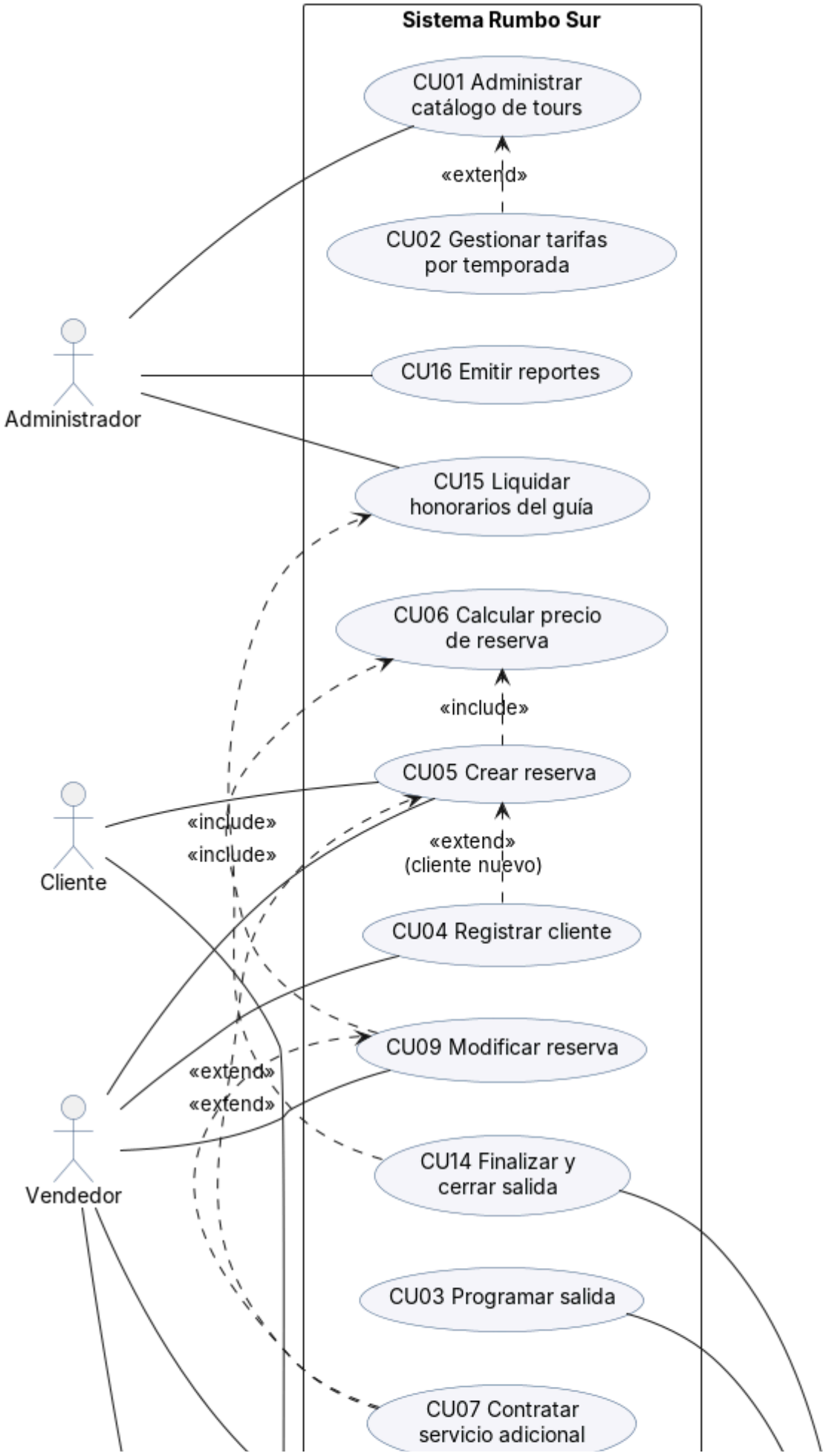
Código	Supuesto
S01	El cliente no opera el sistema: el vendedor carga todo en su nombre.
S02	El titular de la reserva no es necesariamente pasajero; si viaja, se carga también como pasajero.
S03	El tipo de pasajero se calcula a partir de la fecha de nacimiento a la fecha de inicio de la salida (menor de 3, menor de 12, adulto). «Jubilado» es una marca declarada sobre un adulto.

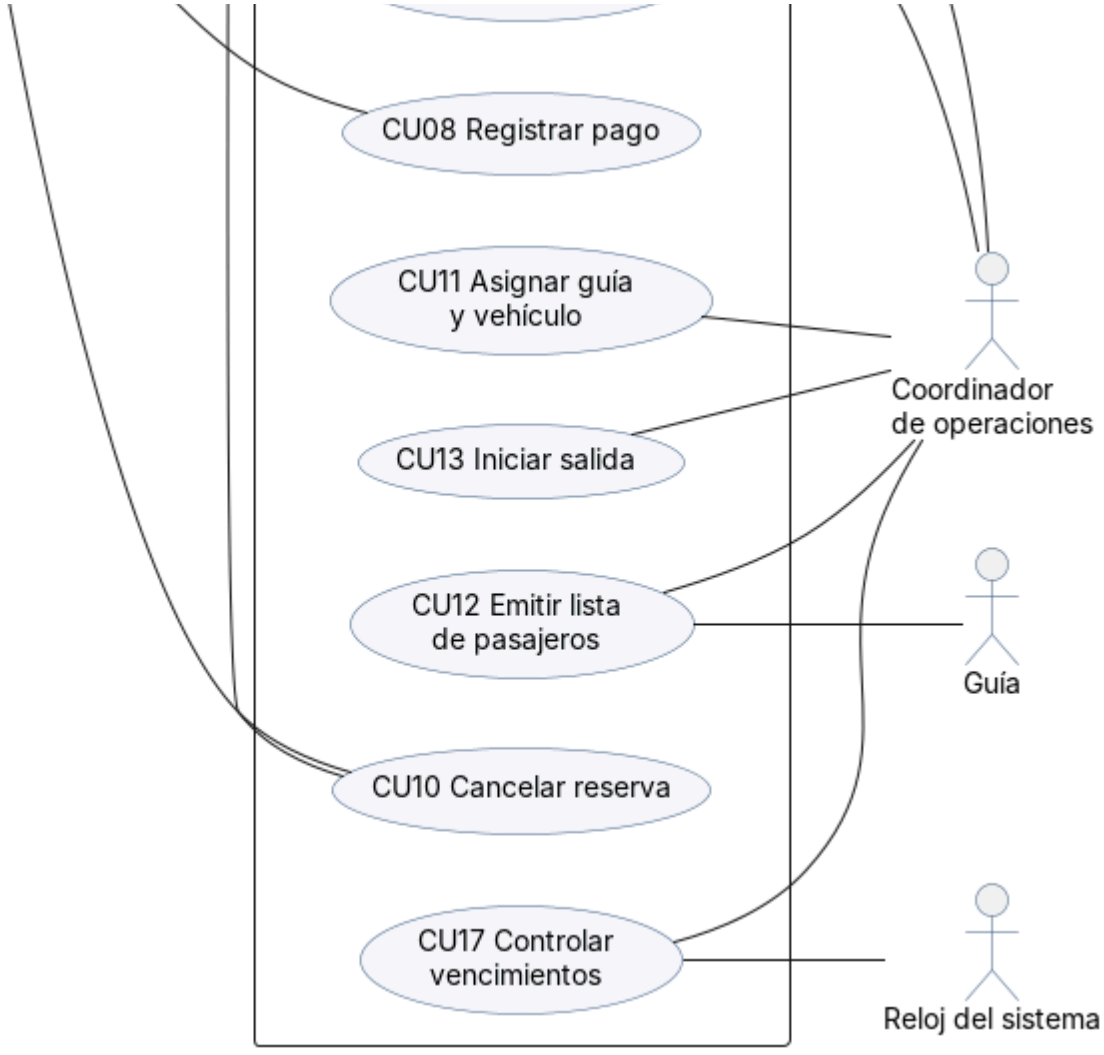
Código	Supuesto
S04	El descuento de jubilado (10 %) se aplica sobre la tarifa de adulto y sí se combina con la promoción, porque es una condición del pasajero y no una promoción.
S05	La promoción se aplica sobre la tarifa base de los pasajeros, no sobre los adicionales.
S06	La temporada se determina por la fecha de inicio de la salida, tomando la tarifa vigente a esa fecha.
S07	El recargo del 7 % se aplica solo sobre el importe del pago hecho con tarjeta en más de una cuota.
S08	Una reserva «pendiente» retiene sus lugares durante 48 h; si en ese plazo no se paga la seña, se anula y libera el cupo.
S09	Los días para la política de cancelación se cuentan en días corridos entre la fecha de cancelación y la fecha de inicio de la salida.
S10	Si una salida finaliza sin adicionales vendidos, el honorario es solo el monto fijo por día. Días trabajados = duración del tour.
S11	Las tareas temporales (cancelación por cupo mínimo, anulación por saldo) se ejecutan al iniciar el programa y a pedido del coordinador, ya que la aplicación no corre de forma continua.
S12	La edad mínima de un tour de aventura se valida contra todos los pasajeros, incluidos los menores de 3 años.
S13	Las devoluciones se registran por el mismo medio del primer pago de la reserva.
S14	La persistencia se hace en archivos CSV, uno por entidad principal, dentro de la carpeta `data/`.

3. Casos de uso

El sistema tiene 17 casos de uso; «Crear reserva» y «Cancelar reserva» son los que se detallan en secuencia (sección 5).

3.1 Diagrama de casos de uso


Sistema Rumbo Sur
CU01 Administrar catálogo de tours
«extend»
CU02 Gestionar tarifas por temporada
Administrador
CU16 Emitir reportes
CU15 Liquidar honorarios del guía
CU06 Calcular precio de reserva
«include»
CU05 Crear reserva
Cliente
«include»
«include»
«extend»
(cliente nuevo)
CU04 Registrar cliente
CU09 Modificar reserva
«extend»
«extend»
Vendedor
CU14 Finalizar y cerrar salida
CU03 Programar salida
CU07 Contratar servicio adicional



El vendedor concentra la venta (CU04 a CU10), el coordinador la operación de las salidas (CU03, CU11 a CU14, CU17) y el administrador el catálogo, la liquidación y los reportes. «Calcular precio» se incluye en crear y modificar reserva para no duplicar la lógica de tarifas.

Fuente: `/docs/uml/casos-de-uso.puml`.

3.2 Descripción de los casos de uso

CU	Nombre	Actor	Relaciones
CU01	Administrar catálogo de tours	Administrador	—
CU02	Gestionar tarifas por temporada	Administrador	«extend» de CU01
CU03	Programar salida	Coordinador	—
CU04	Registrar cliente	Vendedor	«extend» de CU05 (cliente nuevo)

CU	Nombre	Actor	Relaciones
CU05	Crear reserva	Vendedor	«include» CU06 · «extend» CU07
CU06	Calcular precio de reserva	— (interno)	incluido por CU05 y CU09
CU07	Contratar servicio adicional	Vendedor	«extend» de CU05 y CU09
CU08	Registrar pago	Vendedor	—
CU09	Modificar reserva	Vendedor	«include» CU06
CU10	Cancelar reserva	Vendedor	—
CU11	Asignar guía y vehículo	Coordinador	—
CU12	Emitir lista de pasajeros	Coordinador · Guía (secundario)	—
CU13	Iniciar salida	Coordinador	—
CU14	Finalizar y cerrar salida	Coordinador	«include» CU15
CU15	Liquidar honorarios del guía	Administrador	incluido por CU14
CU16	Emitir reportes	Administrador	—
CU17	Controlar vencimientos	Reloj del sistema	—

CU01 – Administrar catálogo de tours

- **Actor:** Administrador. **Precondición:** el administrador ingresó al sistema.
- **Flujo principal:** 1. Elige alta de tour. 2. Ingresa código, nombre, descripción, duración, categoría, cupo mínimo y máximo (si es aventura, nivel de dificultad y edad mínima). 3. Carga las actividades del itinerario (día, orden, horario, descripción, lugar, servicio de proveedor opcional). 4. Define los servicios adicionales del tour. 5. El sistema valida y guarda el tour.
- **Alternativos:** 2a. Código repetido → el sistema informa el error. 2b. Cupo mínimo mayor al máximo → error. 4a. Tour de aventura sin seguro de viaje → el sistema lo agrega como obligatorio. 1a. Modificación o baja: un tour con salidas programadas no se puede dar de baja.

CU02 – Gestionar tarifas por temporada

- **Actor:** Administrador. **Precondición:** el tour existe.
- **Flujo principal:** 1. Selecciona el tour. 2. Ingresa temporada, fecha de vigencia desde/hasta y precio base. 3. El sistema guarda la tarifa.
- **Alternativos:** 2a. El período se superpone con otra tarifa del mismo tour → error.

CU03 – Programar salida

- **Actor:** Coordinador. **Precondición:** el tour tiene una tarifa vigente para la fecha.
- **Flujo principal:** 1. Selecciona el tour. 2. Ingresa fecha de inicio y cupo máximo. 3. El sistema calcula la fecha de fin según la duración. 4. Crea la salida en estado «programada».
- **Alternativos:** 2a. Cupo fuera del rango mínimo–máximo del tour → error. 2b. Fecha pasada → error. 3a. No hay tarifa vigente → error.

CU04 – Registrar cliente

- **Actor:** Vendedor. **Precondición:** ninguna.
- **Flujo principal:** 1. Ingresa nombre, apellido, documento, teléfono, correo y país. 2. El sistema valida y guarda el cliente.
- **Alternativos:** 1a. Documento ya registrado → el sistema muestra el cliente existente.

CU05 – Crear reserva

- **Actor:** Vendedor. **Precondición:** la salida existe y está «programada» o «confirmada»; el cliente está registrado.
- **Flujo principal:** 1. Selecciona salida y cliente. 2. Carga cada pasajero (nombre, documento, fecha de nacimiento, jubilado, contacto de emergencia, restricciones). 3. El sistema valida cupo y condiciones del tour. 4. Opcionalmente contrata adicionales (CU07). 5. El sistema calcula el total (CU06). 6. Registra la reserva «pendiente» asociada al vendedor y descuenta los lugares. 7. Si la salida llega al cupo mínimo pasa a «confirmada»; si llega al máximo, a «completa».
- **Alternativos:** 3a. No hay lugares suficientes → `CupoInsuficienteException`, no se crea la reserva. 3b. Un pasajero no cumple la edad mínima de un tour de aventura → error. 3c. La salida está completa, en curso o cancelada → error. 1a. Cliente no registrado → CU04.

CU06 – Calcular precio de reserva

- **Actor:** interno (incluido). **Precondición:** la reserva tiene salida y pasajeros.

- **Flujo principal:** 1. Obtiene la tarifa vigente por fecha de salida. 2. Para cada pasajero aplica el factor por tipo (adulto 100 %, menor 60 %, infante 0 %) y el descuento de jubilado. 3. Evalúa las promociones aplicables y elige la de mayor descuento. 4. Suma los adicionales por pasajero. 5. Devuelve el total.
- **Alternativos:** 3a. Ninguna promoción aplica → sin descuento.

CU07 – Contratar servicio adicional

- **Actor:** Vendedor. **Precondición:** la reserva está en carga o modificación.
- **Flujo principal:** 1. Elige un pasajero. 2. Elige un servicio del tour. 3. El sistema lo agrega y recalcula el total.
- **Alternativos:** 3a. El seguro obligatorio no puede quitarse en tours de aventura.

CU08 – Registrar pago

- **Actor:** Vendedor. **Precondición:** la reserva está «pendiente» o «confirmada».
- **Flujo principal:** 1. Ingresa importe, medio y cuotas. 2. El sistema aplica el recargo del 7 % si es tarjeta en cuotas. 3. Registra el pago. 4. Si lo pagado alcanza el 30 %, la reserva pasa a «confirmada». 5. Muestra el saldo.
- **Alternativos:** 1a. El importe supera el saldo → error. 1b. Menos de 10 días para la salida y el pago no cubre el saldo → error.

CU09 – Modificar reserva

- **Actor:** Vendedor. **Precondición:** reserva «pendiente» o «confirmada».
- **Flujo principal:** 1. Selecciona la reserva. 2. Agrega o reemplaza un pasajero. 3. El sistema valida condiciones y cupo. 4. Recalcula el total (CU06).
- **Alternativos:** 2a. Reemplazo a menos de 3 días de la salida → error. 3a. El nuevo pasajero no cumple las condiciones del tour → error. 4a. La reserva ya tiene pagos → la diferencia se registra como nota de ajuste.

CU10 – Cancelar reserva

- **Actor:** Vendedor. **Precondición:** reserva «pendiente» o «confirmada».
- **Flujo principal:** 1. Selecciona la reserva e indica el motivo. 2. El sistema calcula los días hasta la salida. 3. Aplica el porcentaje de devolución de la política del cliente. 4. Registra la cancelación (fecha, motivo, importe devuelto) por el mismo medio de pago. 5. Anula la reserva y libera los lugares en la salida.
- **Alternativos:** 3a. La cancelación la provoca la agencia → devolución del 100 %. 1a. Reserva finalizada o ya anulada → `EstadoInvalidoException`. 3b. Sin pagos → devolución 0, solo se anula.

CU11 – Asignar guía y vehículo

- **Actor:** Coordinador. **Precondición:** la salida está «programada», «confirmada» o «completa».
- **Flujo principal:** 1. Selecciona la salida. 2. Elige un guía. 3. El sistema valida categoría habilitada y que no tenga salidas superpuestas. 4. Elige un vehículo. 5. El sistema valida capacidad ≥ cupo. 6. Guarda la asignación.
- **Alternativos:** 3a. Guía no habilitado o con superposición → `AsignacionInvalidaException`. 5a. Capacidad insuficiente → error.

CU12 – Emitir lista de pasajeros

- **Actor:** Coordinador; el Guía la recibe. **Precondición:** la salida tiene reservas confirmadas.
- **Flujo principal:** 1. Selecciona la salida. 2. El sistema arma la lista ordenada por apellido con documento, contacto de emergencia y restricciones. 3. La guarda en archivo y marca la salida con lista emitida.
- **Alternativos:** 2a. No hay reservas confirmadas → aviso.

CU13 – Iniciar salida

- **Actor:** Coordinador. **Precondición:** salida «confirmada» o «completa».
- **Flujo principal:** 1. Selecciona la salida. 2. El sistema verifica guía asignado y lista emitida. 3. Pasa la salida a «en curso».
- **Alternativos:** 2a. Falta guía o lista → error y la salida no cambia de estado.

CU14 – Finalizar y cerrar salida

- **Actor:** Coordinador. **Precondición:** salida «en curso».
- **Flujo principal:** 1. Selecciona la salida. 2. El sistema la pasa a «finalizada». 3. Marca las reservas como finalizadas. 4. Ejecuta CU15.

CU15 – Liquidar honorarios del guía

- **Actor:** Administrador (incluido por CU14). **Precondición:** salida «finalizada».
- **Flujo principal:** 1. Calcula días trabajados × honorario diario. 2. Suma 5 % de las ventas de adicionales de la salida. 3. Registra la liquidación.
- **Alternativos:** 1a. La salida ya fue liquidada → error.

CU16 – Emitir reportes

- **Actor:** Administrador. **Precondición:** existen datos cargados.

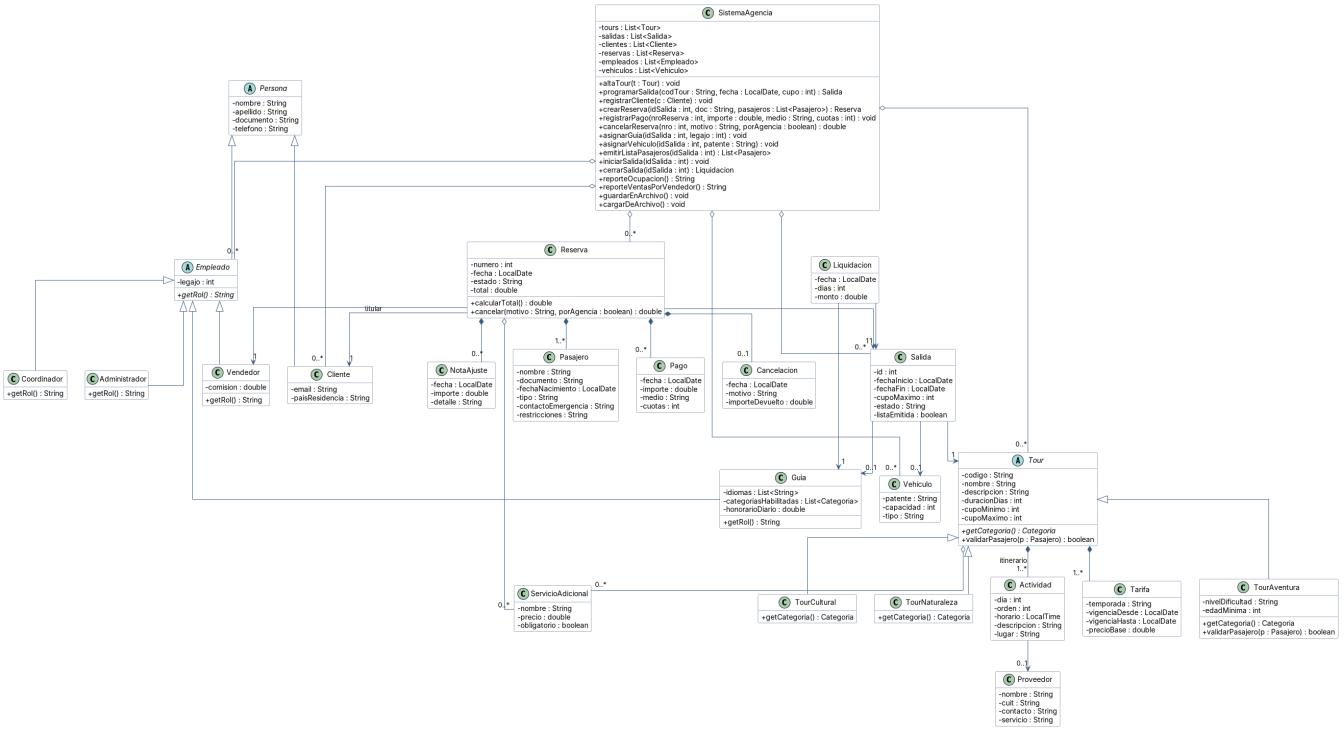
- **Flujo principal:** 1. Elige el reporte (ocupación por salida, ventas por vendedor, ingresos por tour o cancelaciones) y el período. 2. El sistema lo calcula, lo muestra ordenado y lo exporta a archivo.
- **Alternativos:** 1a. Sin datos en el período → reporte vacío.

CU17 – Controlar vencimientos

- **Actor:** Reloj del sistema (al iniciar) o Coordinador a pedido. **Precondición:** ninguna.
- **Flujo principal:** 1. Busca salidas «programadas» a 7 días o menos sin cupo mínimo y las cancela con devolución del 100 %. 2. Busca reservas con saldo impago a 10 días o menos de la salida y las anula liberando cupo. 3. Anula reservas «pendientes» sin seña tras 48 h.

4. Diagrama de clases inicial

La primera versión modela correctamente el dominio, pero concentra el comportamiento en una clase `SistemaAgencia` y resuelve precios y cancelaciones con condicionales. Esos problemas son los que corrige la sección 6.



Fuente: `/docs/uml/clases-inicial.puml`.

4.1 Jerarquías de herencia

Jerarquía	Clase base	Subclases	Método sobrescrito
Personas	`Persona` (abstracta)	`Cliente`, `Empleado` (abstracta) → `Vendedor`, `Guia`, `Coordinador`, `Administrador`	`getRol()` (abstracto en `Empleado`)
Tours	`Tour` (abstracta)	`TourAventura`, `TourCultural`, `TourNaturaleza`	`getCategoria()` (abstracto) y `validarPasajero()` (aventura agrega edad mínima)

4.2 Relaciones principales

Relación	Tipo	Multiplicidad	Motivo
Tour – Actividad	Composición	1 – 1..*	El itinerario no existe sin el tour.
Tour – Tarifa	Composición	1 – 1..*	Cada tarifa es de un tour y una temporada.
Tour – ServicioAdicional	Agregación	1 – 0..*	El servicio (p. ej. seguro) se reutiliza entre tours.
Actividad – Proveedor	Asociación	0..1	Solo algunas actividades incluyen servicios de terceros.
Salida – Tour / Guia / Vehiculo	Asociación	1 / 0..1 / 0..1	Guía y vehículo se asignan después de programar.
Reserva – Pasajero, Pago, Cancelacion, NotaAjuste	Composición	1..* / 0..* / 0..1 / 0..*	Viven y se persisten con la reserva.
Reserva – Cliente / Vendedor / Salida	Asociación	1 / 1 / 1	Titular, quién vendió y para qué salida.
Liquidacion – Guia / Salida	Asociación	1 / 1	Honorario de un guía por una salida finalizada.

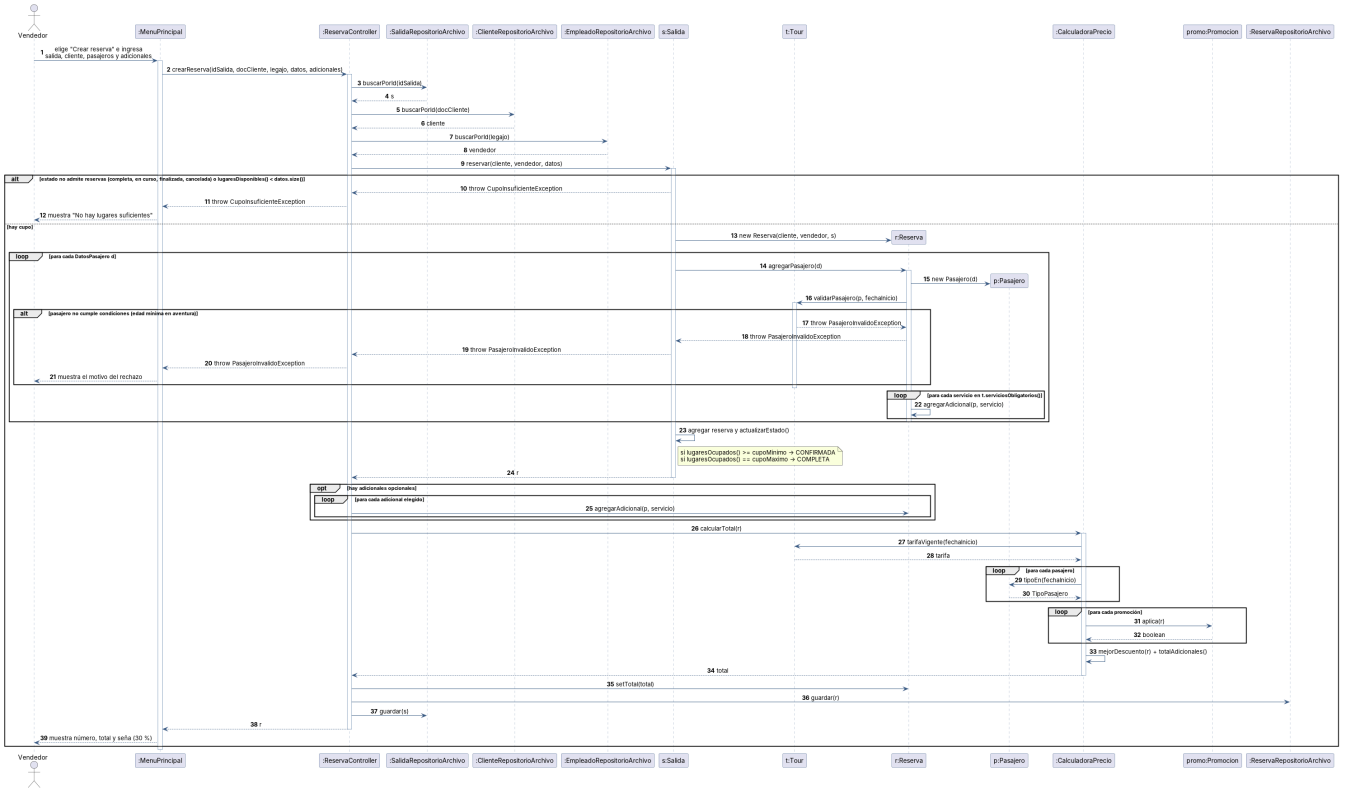
4.3 Problemas detectados en esta versión

1. **Clase dios:** `SistemaAgencia` tiene 15 operaciones de áreas distintas (catálogo, ventas, operación, reportes y archivos).
2. **Precio rígido:** `Reserva.calcularTotal()` decide con `if` el tipo de pasajero y las dos promociones; una promoción nueva obliga a modificar `Reserva`.
3. **Cancelación con bandera:** `cancelar(motivo, porAgencia)` mezcla dos políticas de devolución en una cadena de condicionales.
4. **Persistencia acoplada:** la lógica de negocio llama directamente a `guardarEnArchivo()`; no se puede probar sin tocar disco.
5. **Responsabilidades fuera de lugar:** el cupo disponible, la disponibilidad del guía y la creación de pagos se calculan en `SistemaAgencia` y no en quien tiene los datos.
6. **Estados como `String`:** permiten valores inválidos y transiciones no permitidas.

5. Diagramas de secuencia

Se eligieron **CU05 Crear reserva** (dos flujos de error: cupo insuficiente y pasajero inválido) y **CU10 Cancelar reserva** (reserva inexistente, estado inválido y dos políticas de devolución). Ambos usan las clases del diagrama final (sección 7), que es el que se implementa en la Etapa 2.

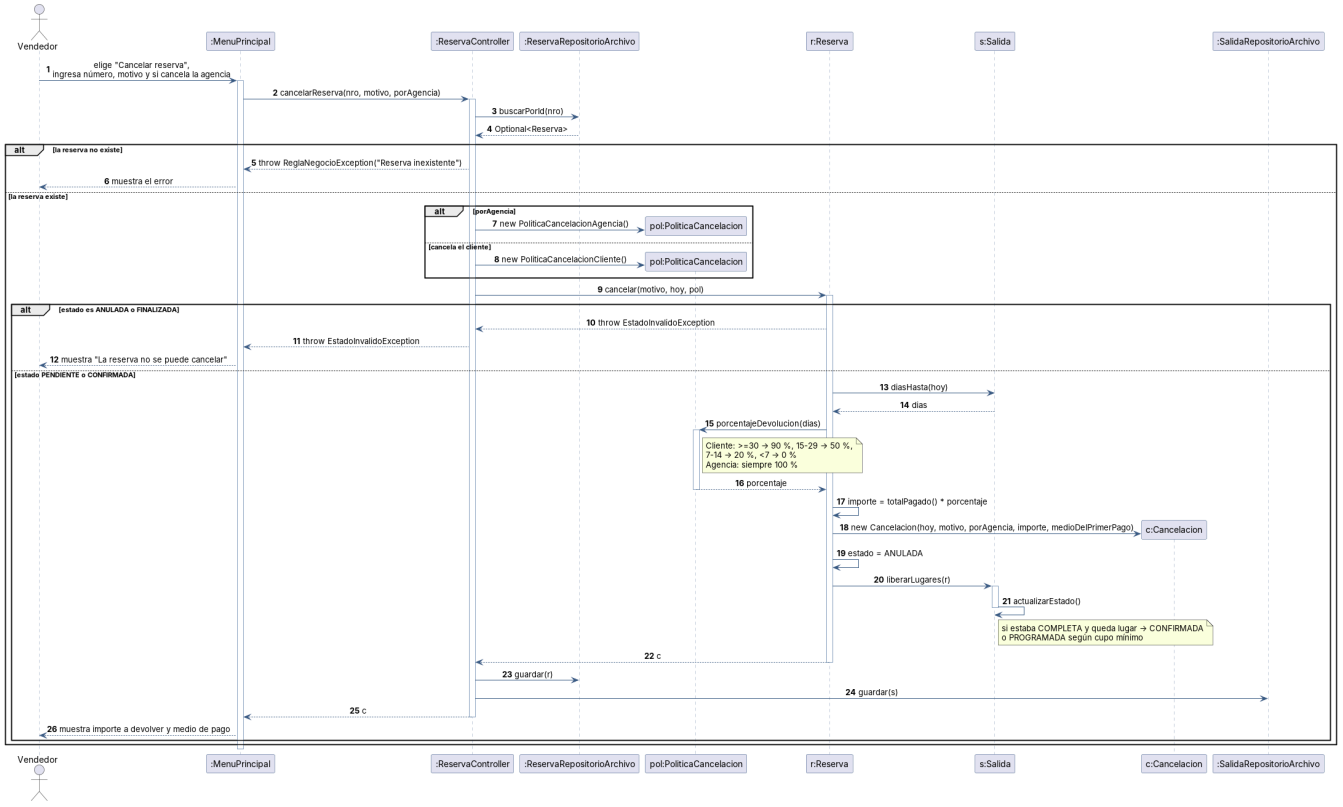
5.1 CU05 – Crear reserva



Fuente: `/docs/uml/secuencia-crear-reserva.puml`.

- **Controlador:** `ReservaController` recibe el pedido de la interfaz y coordina; no calcula ni valida reglas.
- **Creador:** `Salida` crea la `Reserva` porque la contiene, y `Reserva` crea cada `Pasajero`.
- **Fragmento `alt` (error):** si la salida no admite reservas o `lugaresDisponibles()` es menor que la cantidad de pasajeros, se lanza `CupoInsuficienteException` y no se crea nada. Un segundo `alt` rechaza al pasajero que no cumple la edad mínima de un tour de aventura.
- **Fragmentos `loop` y `opt`:** se recorren los pasajeros, se agregan los servicios obligatorios (seguro en aventura) y, opcionalmente, los adicionales elegidos.
- **Precio:** `CalculadoraPrecio` pide la tarifa vigente al `Tour`, el tipo a cada `Pasajero` y consulta cada `Promocion`; se aplica la de mayor descuento.

5.2 CU10 – Cancelar reserva



Fuente: `/docs/uml/secuencia-cancelar-reserva.puml`.

- **Fragmento `alt` externo:** si la reserva no existe, el controlador lanza `ReglaNegocioException`.
- **Selección de política:** el controlador crea `PoliticaCancelacionAgencia` (100 %) o `PoliticaCancelacionCliente` (escala 90/50/20/0 %); `Reserva` solo conoce la interfaz.

- **Fragmento `alt` interno:** una reserva anulada o finalizada lanza `EstadoInvalidoException`.
- **Experto:** `Reserva` calcula `totalPagado()` y crea la `Cancelacion`; `Salida` calcula `diasHasta(hoy)`, libera los lugares y recalcula su estado.

6. Aplicación de patrones: 3 SOLID y 4 GRASP

Se aplicaron **SRP, OCP y DIP** de SOLID y **Experto, Creador, Controlador y Polimorfismo** de GRASP. Cada uno parte de un problema concreto de la sección 4.3.

6.1 SOLID

S – Principio de Responsabilidad Única (SRP)

- **Problema:** `SistemaAgencia` cambiaba por cualquier motivo: una regla de precio, un reporte nuevo o el formato del archivo. `Reserva` además calculaba precios y promociones.
- **Decisión:** dividir `SistemaAgencia` en cuatro controladores por área y sacar el cálculo de precio a `CalculadoraPrecio`. La persistencia pasa a repositorios, uno por entidad.
- **Dónde se ve:** `ReservaController`, `SalidaController`, `CatalogoController`, `ReporteController`, `CalculadoraPrecio`, `ReservaRepositorioArchivo` y demás repositorios.
- **Ventaja:** cada clase tiene un solo motivo de cambio; las pruebas unitarias de precio no dependen de archivos ni de la interfaz, y dos integrantes pueden trabajar en paralelo sin conflictos en el mismo archivo.

O – Principio Abierto/Cerrado (OCP)

- **Problema:** agregar una promoción (p. ej. «cliente frecuente») o una política de cancelación obligaba a editar los `if` de `Reserva`.
- **Decisión:** definir las interfaces `Promocion` y `PoliticaCancelacion`. `CalculadoraPrecio` recibe una lista de promociones y elige la de mayor descuento; `Reserva.cancelar()` recibe la política.
- **Dónde se ve:** `Promocion` ← `PromocionAnticipada`, `PromocionGrupo`; `PoliticaCancelacion` ← `PoliticaCancelacionCliente`, `PoliticaCancelacionAgencia`.
- **Ventaja:** una promoción nueva es una clase nueva que implementa la interfaz; `CalculadoraPrecio` y `Reserva` no se modifican. La regla «no se acumulan» queda en un único lugar (`mejorDescuento`).

D – Principio de Inversión de Dependencias (DIP)

- **Problema:** la lógica de negocio llamaba directamente a la escritura de archivos, de modo que no se podía probar sin disco ni cambiar el formato sin tocar la lógica.
- **Decisión:** los controladores dependen de la abstracción `Repositorio<T, ID>`; las implementaciones concretas (`RepositorioArchivo` y sus subclases CSV) se inyectan por constructor desde `MenuPrincipal`.
- **Dónde se ve:** `ReservaController(Repositorio<Reserva,Integer>, Repositorio<Salida,Integer>, …, CalculadoraPrecio)`.
- **Ventaja:** en las pruebas unitarias se pasa un repositorio en memoria; cambiar CSV por otro formato no toca los controladores.

6.2 GRASP

Experto en información

- **Problema:** `SistemaAgencia` contaba los lugares ocupados recorriendo reservas, sumaba pagos y verificaba la disponibilidad del guía, con datos que no le pertenecían.
- **Decisión:** cada cálculo lo hace la clase que tiene los datos.
- **Dónde se ve:** `Salida.lugaresDisponibles()`, `Salida.diasHasta()`, `Salida.ventasAdicionales()`; `Reserva.cantidadLugares()`, `totalPagado()`, `saldo()`; `Tour.tarifaVigente(fecha)`; `Guia.puedeGuiar(tour)` y `estaDisponible(...)`; `Pasajero.tipoEn(fecha)`; `Liquidacion.total()`.
- **Ventaja:** encapsulamiento real (sin getters de listas internas) y menos acoplamiento; si cambia cómo se cuenta un lugar, cambia una sola clase.

Creador

- **Problema:** `SistemaAgencia` creaba reservas, pasajeros y pagos y luego los «enganchaba», con riesgo de objetos inconsistentes (una reserva sin salida, un pago sin reserva).
- **Decisión:** crea el objeto quien lo contiene o lo registra.
- **Dónde se ve:** `Salida.reservar(...)` crea la `Reserva` (la contiene y conoce el cupo); `Reserva.agregarPasajero(datos)` crea el `Pasajero`; `Reserva.registrarPago(...)` crea el `Pago` y aplica el recargo; `Reserva.cancelar(...)` crea la `Cancelacion`; `Salida.finalizar()` crea la `Liquidacion`.
- **Ventaja:** las reglas de creación (cupo, edad mínima, seña del 30 %) se validan en el mismo punto donde nace el objeto.

Controlador

- **Problema:** la interfaz de consola debía hablar con una única clase gigante o, peor, manipular el modelo directamente.
- **Decisión:** un controlador de caso de uso por área recibe los eventos del sistema y delega en el modelo.
- **Dónde se ve:** `MenuPrincipal` solo llama a `ReservaController`, `SalidaController`, `CatalogoController` y `ReporteController`; los mensajes de los diagramas de secuencia empiezan en ellos.
- **Ventaja:** la interfaz se puede cambiar de consola a Swing sin tocar el modelo; los controladores no contienen reglas de negocio, solo coordinan y persisten.

Polimorfismo

- **Problema:** las reglas que varían por tipo (categoría de tour, política de devolución, tipo de pasajero) se resolvían con `if` sobre `String`.
- **Decisión:** que cada tipo responda al mismo mensaje con su propio comportamiento.
- **Dónde se ve:** `TourAventura` sobrescribe `validarPasajero()` (edad mínima) y `serviciosObligatorios()` (seguro); `PoliticaCancelacion.porcentajeDevolucion()` y `Promocion.aplica()` tienen una implementación por clase; `TipoPasajero` es un enum con su factor (0, 0,6 y 1).
- **Ventaja:** el código cliente no pregunta «de qué tipo sos»; agregar una categoría o política no rompe lo existente. Además se cumple la sustitución: cualquier `Tour` puede usarse donde se espera un `Tour`.

6.3 Resumen

Patrón	Problema que resuelve	Clases donde se ve
SRP	Clase dios y precio dentro de `Reserva`	Controladores, `CalculadoraPrecio`, repositorios
OCP	Promociones y políticas en `if`	`Promocion`, `PoliticaCancelacion` e implementaciones
DIP	Lógica acoplada a archivos	`Repositorio<T,ID>`, `RepositorioArchivo`
Experto	Cálculos fuera de quien tiene los datos	`Salida`, `Reserva`, `Tour`, `Guia`, `Pasajero`
Creador	Objetos creados sin su contenedor	`Salida` → `Reserva`; `Reserva` → `Pasajero`, `Pago`, `Cancelacion`

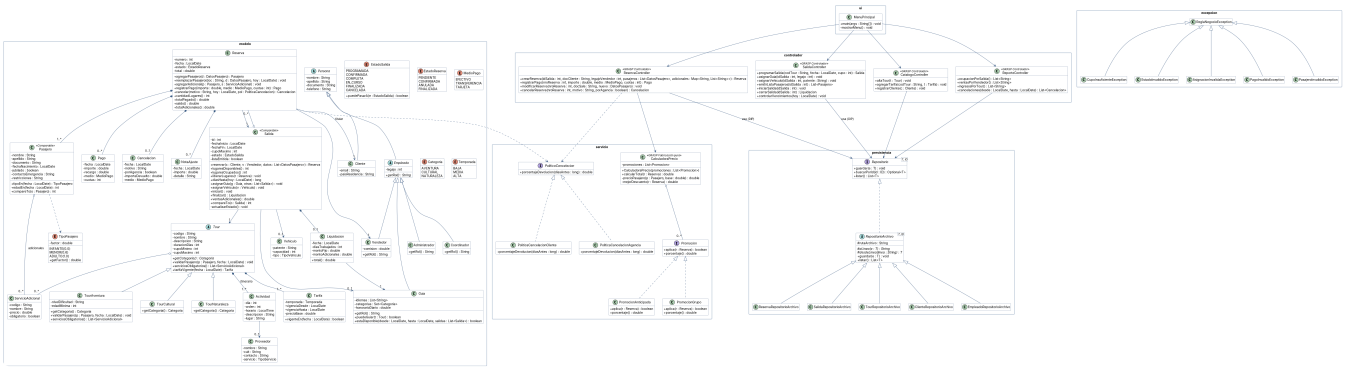
Patrón	Problema que resuelve	Clases donde se ve
Controlador	UI acoplada al modelo	`ReservaController`, `SalidaController`, `CatalogoController`, `ReporteController`
Polimorfismo	`if` por tipo	`Tour` y subclases, `TipoPasajero`, `Promocion`, `PoliticaCancelacion`

Como apoyo, `CalculadoraPrecio` es además una **Fabricación pura**: no existe en el dominio, se inventa para lograr alta cohesión.

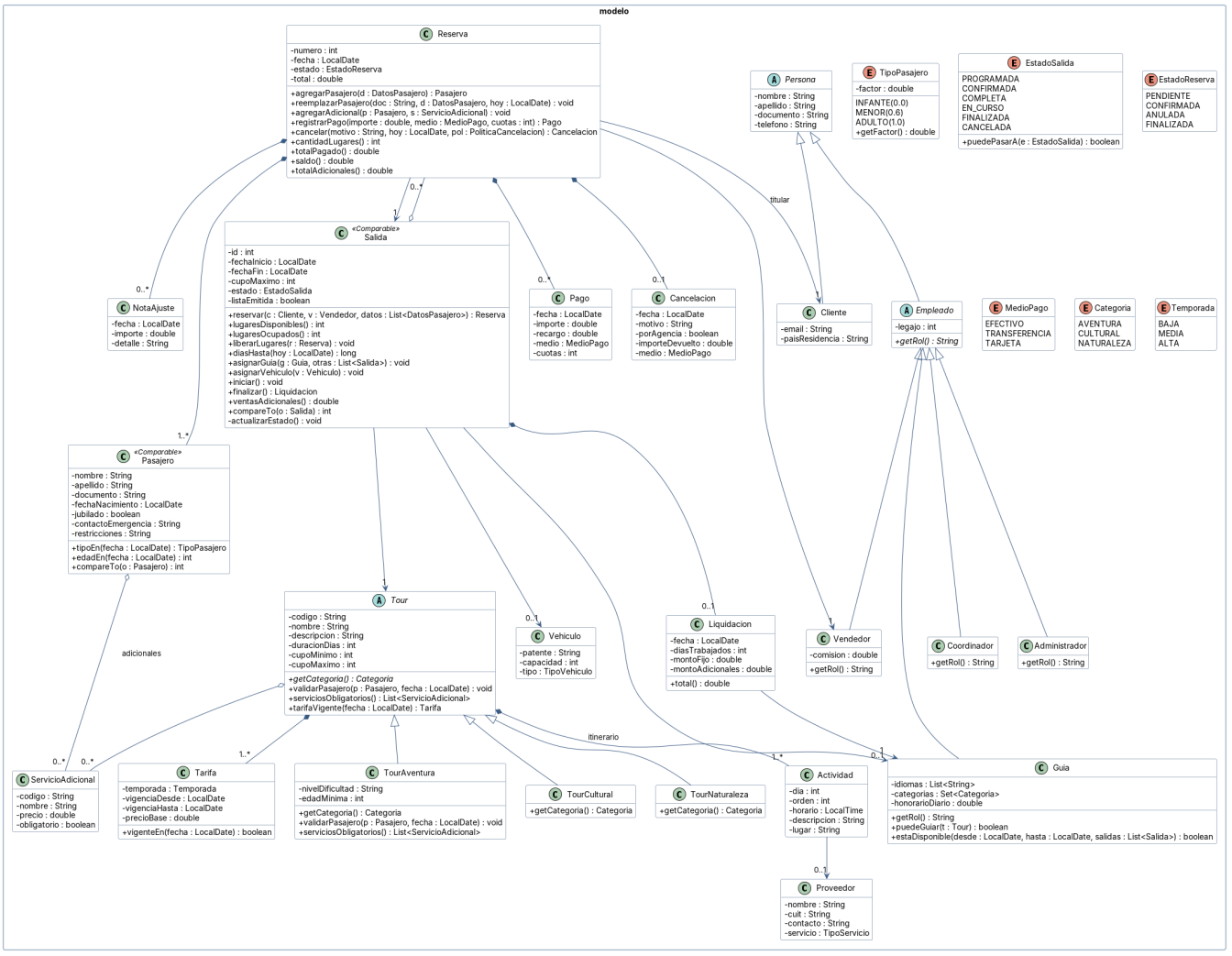
7. Diagrama de clases final

El diseño final reemplaza `SistemaAgencia` por cinco paquetes (`ui`, `controlador`, `servicio`, `persistencia`, `modelo`) más `excepcion`, y es el que se codifica en la Etapa 2. Por su tamaño se muestra completo y en dos vistas legibles.

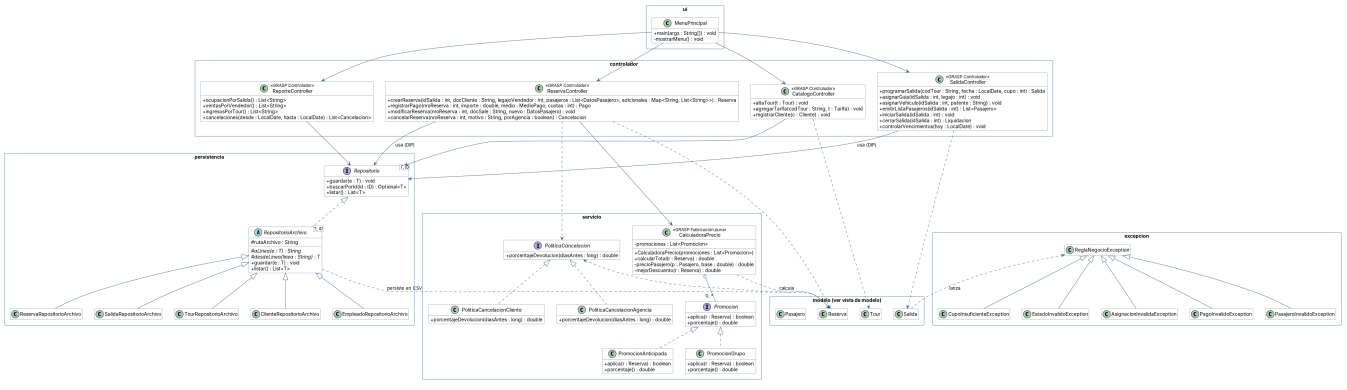
7.1 Vista completa



7.2 Vista del paquete modelo



7.3 Vista de controladores, servicios, persistencia y excepciones



Fuentes: `/docs/uml/clases-final.puml`, `clases-final-modelo.puml` y `clases-final-capas.puml`.

7.4 Nota de cambios respecto del diagrama inicial

#	Cambio	Motivo	Patrón
1	Se elimina `SistemaAgencia`; aparecen `ReservaController`, `SalidaController`, `CatalogoController` y `ReporteController`.	Clase dios con motivos de cambio mezclados.	SRP, Controlador
2	Nueva clase `CalculadoraPrecio`; se quita `calcularTotal()` de `Reserva`.	La reserva no debe conocer tarifas ni promociones.	SRP, Fabricación pura
3	Nueva interfaz `Promocion` con `PromocionAnticipada` y `PromocionGrupo`.	Promociones nuevas sin modificar código existente.	OCP, Polimorfismo
4	Nueva interfaz `PoliticaCancelacion` con implementaciones de cliente y agencia; `cancelar()` recibe la política en vez de un `boolean`.	Eliminar la bandera y la cadena de `if`.	OCP, Polimorfismo
5	Nuevo paquete `persistencia` con `Repositorio<T,ID>`, `RepositorioArchivo` y un repositorio por entidad; se quitan `guardarEnArchivo()` y `cargarDeArchivo()`.	Desacoplar la lógica del almacenamiento y poder probar en memoria.	DIP, SRP
6	Métodos nuevos en el modelo: `Salida.lugaresDisponibles()`, `diasHasta()`, `liberarLugares()`, `ventasAdicionales()`; `Reserva.totalPagado()`, `saldo()`, `cantidadLugares()`; `Tour.tarifaVigente()`; `Guia.puedeGuiar()`, `estaDisponible()`.	Cada cálculo en la clase que tiene los datos.	Experto
7	`Salida.reservar()` crea la reserva; `Reserva` crea `Pasajero`, `Pago` y `Cancelacion`; `Salida.finalizar()` crea la `Liquidacion`.	Validar reglas al momento de crear.	Creador

#	Cambio	Motivo	Patrón
8	`Tour.serviciosObligatorios()` sobrescrito en `TourAventura`; `validarPasajero()` pasa a lanzar excepción.	El seguro obligatorio y la edad mínima dependen de la categoría.	Polimorfismo
9	Los estados y tipos pasan de `String` a enums: `EstadoSalida` (con `puedePasarA()`), `EstadoReserva`, `TipoPasajero` (con factor), `MedioPago`, `Categoria`, `Temporada`.	Evitar valores inválidos y transiciones no permitidas.	Polimorfismo, Experto
10	`Pasajero` reemplaza el atributo `tipo` por `fechaNacimiento` + `jubilado`, y calcula `tipoEn(fecha)`; suma `apellido`.	El tipo depende de la edad a la fecha de salida (S03) y la lista se ordena por apellido.	Experto
11	La relación de adicionales pasa de `Reserva` a `Pasajero`.	El enunciado indica que los adicionales se contratan por pasajero.	—
12	`Salida` y `Pasajero` implementan `Comparable` (por fecha y por apellido); los reportes usan `Comparator`.	Requisito técnico de ordenamiento natural y personalizado.	—
13	Paquete `excepcion` con `ReglaNegocioException` y cinco subclases.	Reglas de negocio informadas con `throw` / `throws`.	—
14	`Liquidacion` pasa a ser parte de `Salida` (composición) y guarda monto fijo y monto de adicionales por separado.	Se liquida una vez por salida finalizada (RN15).	Creador